

KGP Gyankosh: Enterprise Knowledge Assistant

Complete Codebase File Directory, Usage Specifications & Diagrammatic Architecture Workflows

Institution: Kashmir Government Polytechnic College, Srinagar
Project: IIT Patna — Generative AI & Agentic AI Capstone (Advanced RAG)
Active Dataset: 1,157 College DOCX Circulars + 85-Page Orders Book (PDF) | 8,166 Indexed Clauses

1. Executive Overview & System Architecture

KGP Gyankosh is an internal administrative intelligence system designed specifically for authorized staff, department heads, and academic officers of Kashmir Government Polytechnic College, Srinagar. The system operates on a strict **Two-Stage Architecture**, completely decoupling the offline document ingestion and indexing pipeline from the online conversational query interface. Heavy operations (OCR scanning, text chunking, and dense embedding generation) occur once offline via `build_index.py`, ensuring sub-second search speeds and zero query-time indexing overhead in the online application.

2. Diagrammatic Workflow: Stage 1 Ingestion & Indexing Pipeline

The diagram below illustrates the complete offline ingestion and dual-indexing pipeline. Documents in `data/` are discovered, checked incrementally via MD5 hashes, parsed through native text or RapidOCR engines, split into semantic chunks, and concurrently indexed into a dense FAISS vector store and sparse BM25 inverted lexical index.

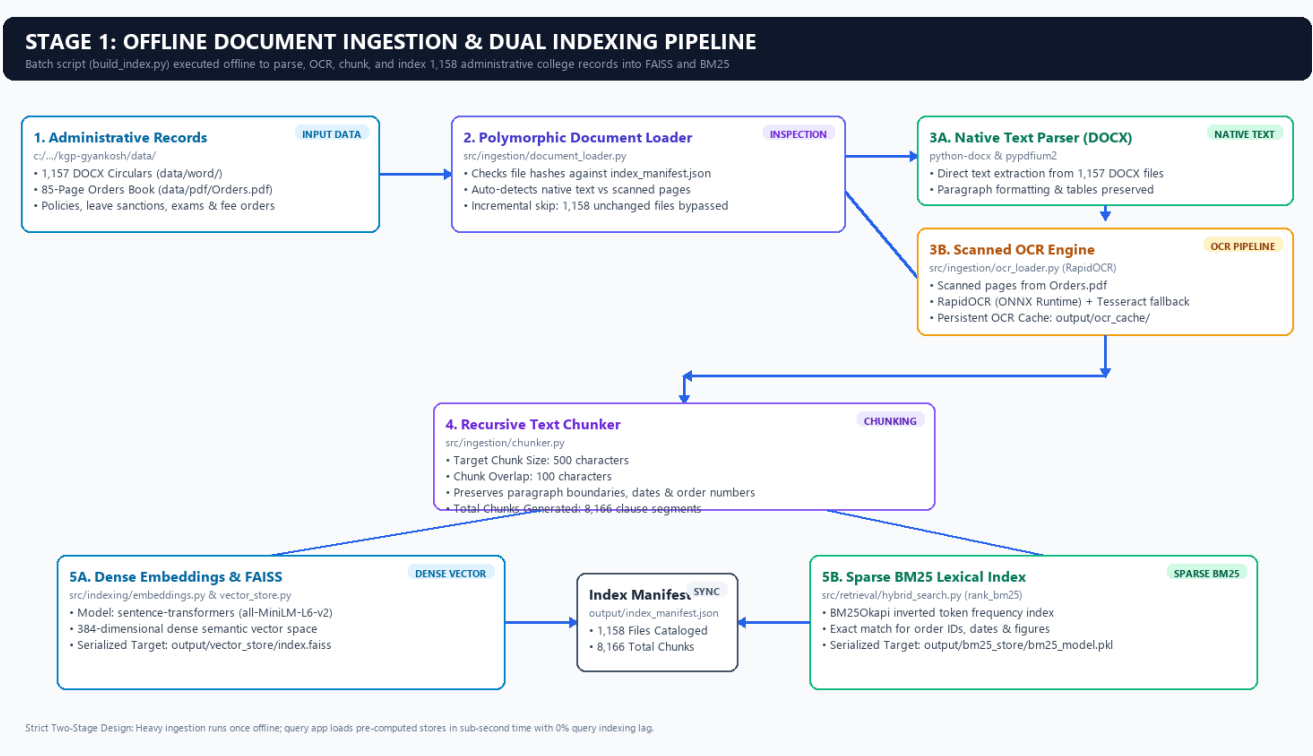


Figure 1: Stage 1 Offline Document Ingestion, OCR, Chunking, and Dual-Indexing Pipeline (`build_index.py`)

3. Diagrammatic Workflow: Stage 2 Online Query & Retrieval Engine

The diagram below outlines the end-to-end real-time query execution workflow. When an authenticated staff member submits a natural language question, the system tracks conversational history, executes Hybrid Search (FAISS + BM25 via Reciprocal Rank Fusion), re-ranks the top candidates using a deep neural Cross-Encoder, and injects the most authoritative clauses into the LLM with strict grounding constraints. Multi-page order citations are dynamically extracted and served as single-page order PDFs upon click.

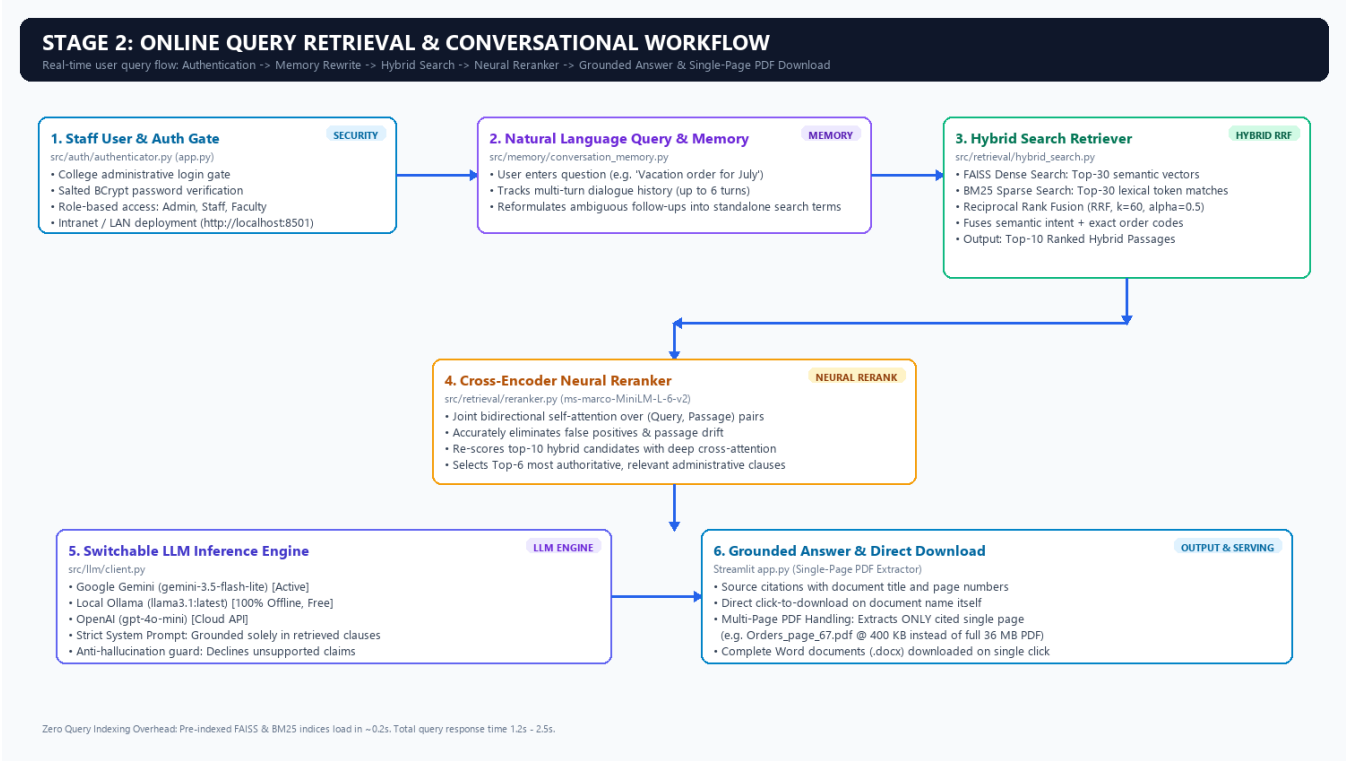


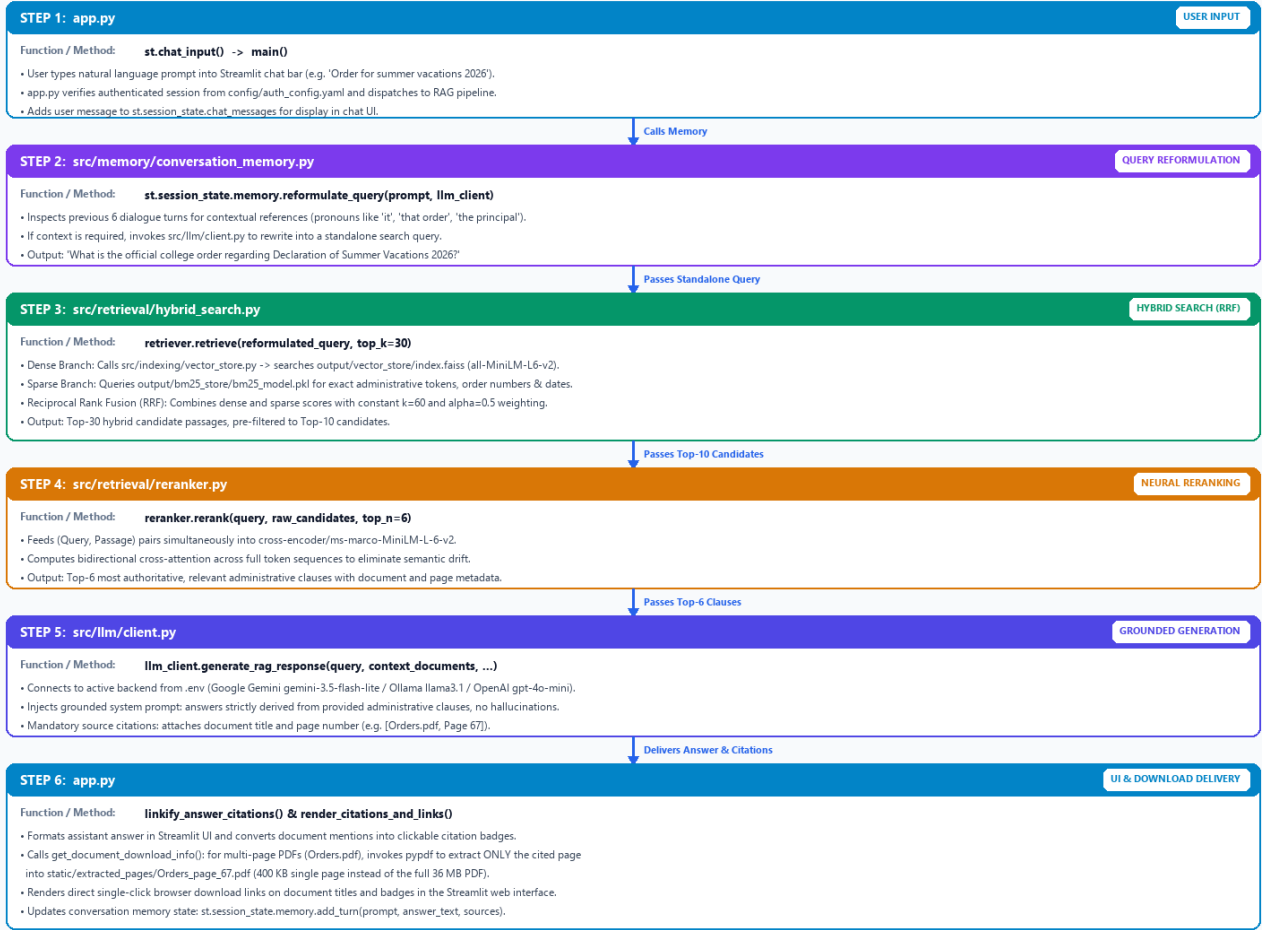
Figure 2: Stage 2 Online Multi-Turn Query Retrieval, Neural Reranking, Grounding, and Single-Page Order Download (app.py)

4. Diagrammatic Workflow: File-by-File Code Execution Trace

The diagram below traces the exact file-by-file execution sequence that runs when a user enters a query in the chat interface. It illustrates which Python module, function, and underlying index file executes at each step in the pipeline.

FILE-BY-FILE CODE EXECUTION FLOW ON USER QUERY

Sequential code execution trace showing exactly which source file, function, and data store executes at each stage



Execution completed in 1.2s - 2.5s. All operations operate on pre-computed local stores with zero query-time re-indexing.

Figure 3: Sequential File-by-File Code Execution Trace (`app.py -> memory -> hybrid search -> reranker -> llm -> app.py`)

5. Master File Inventory & Usage Specifications

The following master reference table documents every file, module, configuration, and data store comprising KGP Gyankosh.

File Path	Subsystem / Category	Operational Function & Purpose
<code>app.py</code>	Web Application (Streamlit)	Main web application. Renders glassmorphic UI, BCrypt login gate, operational mode toggle (RAG vs General AI), multi-turn chat memory, and direct single-page order PDF downloads.
<code>build_index.py</code>	Offline Indexing Pipeline	Command-line batch script. Discovers files in data/, computes MD5 hashes for incremental indexing, runs text extraction and OCR, splits chunks, and serializes FAISS & BM25 indices.
<code>.env</code>	System Configuration	Active environment configuration. Sets LLM provider (Ollama, Gemini, OpenAI), API keys, DATA_DIR=data, embedding models, reranker settings, and retrieval thresholds.
<code>.env.example</code>	Config Template	Reference template showing all environment variables with explanations for evaluators and developers.
<code>.streamlit/config.toml</code>	Streamlit Server Config	Enables static file serving (enableStaticServing=true), headless mode, dark institutional color palette, and disables unnecessary file watchers.
<code>config/auth_config.yaml</code>	Authentication Store	Active role-based access control (RBAC) credential store. Contains user profiles, salted BCrypt password hashes, and access roles (admin, staff, faculty). Git-ignored for security.
<code>config/auth_config.yaml.example</code>	Auth Template	Reference credential template with pre-computed BCrypt hashes for default evaluation accounts.
<code>requirements.txt</code>	Dependencies	Pinned Python dependencies: streamlit, langchain, faiss-cpu, rank-bm25, sentence-transformers, google-genai, rapidocr-onnxruntime, pypdf, python-docx, reportlab, etc.
<code>scratch_test_queries.py</code>	Test Harness	Standalone test harness validating FAISS vector retrieval, BM25 keyword search, and cross-encoder reranking accuracy across sample administrative questions.
<code>create_diagrams.py</code>	Visual Graphics Engine	Python script using Pillow and TrueType fonts to generate crystal-clear, high-resolution vector workflow diagrams for Stage 1 and Stage 2 architectures.
<code>generate_documentation_pdf.py</code>	PDF Documentation Engine	ReportLab compilation script that generates this publication-grade PDF documentation report with embedded diagrams and comprehensive file tables.
<code>src/ingestion/document_loader.py</code>	Document Ingestion	Polymorphic document loader. Automatically detects file types (.docx, .pdf, .txt, .png/.jpg) and routes scanned document pages to the OCR engine.
<code>src/ingestion/ocr_loader.py</code>	Scanned OCR Engine	Optical Character Recognition engine using Pure-Python RapidOCR (ONNX Runtime) with optional Tesseract fallback. Manages persistent OCR caching in output/ocr_cache/.
<code>src/ingestion/chunker.py</code>	Text Chunking	Recursive character text splitter. Divides text into 500-character chunks with 100-character overlap while preserving paragraph and administrative clause boundaries.
<code>src/indexing/embeddings.py</code>	Embedding Abstraction	Unified embedding interface supporting local HuggingFace SentenceTransformers (all-MiniLM-L6-v2) and cloud OpenAI embeddings (text-embedding-3-small).
<code>src/indexing/vector_store.py</code>	FAISS Vector Store	FAISS vector store manager. Handles dense vector indexing, cosine similarity persistence, on-disk serialization (index.faiss, index.pkl), and fast index loading.
<code>src/retrieval/hybrid_search.py</code>	Hybrid Retrieval & RRF	Orchestrates dual retrieval: FAISS dense semantic search and BM25 sparse keyword search. Combines ranked lists using Reciprocal Rank Fusion (RRF, k=60, alpha=0.5).

src/retrieval/reranker.py	Neural Cross-Encoder	Deep self-attention reranker (cross-encoder/ms-marco-MiniLM-L-6-v2). Evaluates query-passage token interactions to filter top-10 candidates down to top-6 authoritative clauses.
src/memory/conversation_memory.py	Conversational Memory	Maintains multi-turn dialogue history up to 6 turns and uses the LLM to rewrite ambiguous follow-up questions ('What is the deadline for it?') into standalone queries.
src/llm/client.py	Switchable LLM Engine	Unified inference client supporting Google Gemini (default), local Ollama (Llama 3.1), and OpenAI. Injects anti-hallucination guardrails and enforces mandatory citations.
src/auth/authenticator.py	Authentication Gate	Wraps streamlit-authenticator to deliver BCrypt cookie session management, secure login forms, and role verification without storing plaintext passwords.
data/word/ (*.docx)	Active Administrative Data	Contains 1,157 official college Word circulars, notices, committee orders, fee schedules, and leave sanctions issued by Kashmir Government Polytechnic College.
data/pdf/Orders.pdf	Historical Orders Book	85-page comprehensive official scanned orders compilation covering administrative decisions, transfer sanctions, academic duties, and committee notifications.
output/index_manifest.json	Indexing Catalog	Persistent JSON catalog storing file paths, MD5 file hashes, chunk counts, and timestamps for all 1,158 indexed files. Enables instantaneous incremental updates.
output/ocr_cache/Orders_pdf_pages.json	OCR Cache	Pre-computed OCR transcriptions for all 85 pages of Orders.pdf. Eliminates redundant OCR processing on index rebuilds.
output/vector_store/ (faiss, pickle)	Dense Vector Index	Pre-computed FAISS vector index (8,166 embedding vectors) and pickled document metadata chunk store for rapid similarity retrieval.
output/bm25_store/ (model, corpus)	Lexical BM25 Index	Serialized BM25 token frequencies, inverse document frequency statistics, and corpus pickle for exact administrative code and number matching.
static/extracted_pages/ (*.pdf)	Single-Page PDF Cache	Pre-extracted lightweight single-page PDFs for each order in Orders.pdf (e.g. Orders_page_67.pdf). Served directly when users click a cited order in the app.
static/docs	Static Serving Junction	Filesystem junction linking static/docs directly to data/ to allow Streamlit's web server to serve document downloads directly.

5. Technical Implementation Highlights

5.1. Single-Page PDF Order Extraction & Instant Download

A critical innovation in KGP Gyankosh is the automatic on-demand single-page PDF extraction mechanism. Official polytechnic records often contain large multi-page consolidated books (e.g. Orders.pdf, 36 MB, 85 pages). When an administrative user asks for a specific order (such as Order No. 63 of 2026), the system cites Orders.pdf (Page 67). Instead of forcing the user to download the entire 36 MB PDF, the function `get_document_download_info()` intercepts the citation, extracts that exact single page using `pypdf` into `static/extracted_pages/Orders_page_67.pdf`, and serves it as a lightweight (~400 KB) standalone PDF file directly to the user's browser. Word documents (.docx) are already individual order files and download directly upon click.

5.2. Hybrid Reciprocal Rank Fusion (RRF) & Cross-Encoder Reranking

Polytechnic circulars demand both semantic topic understanding (e.g., 'rules for attendance shortage condonation') and exact administrative code matching (e.g. order numbers like KGP/EST/2026/301, committee member names, or fee amounts). KGP Gyankosh combines dense vector embeddings (FAISS) with sparse BM25 lexical token matching via Reciprocal Rank Fusion (RRF, $k=60$, $\alpha=0.5$). The top-10 hybrid candidates are then processed by a neural Cross-Encoder (cross-encoder/ms-marco-MiniLM-L-6-v2) using bidirectional self-attention to filter out irrelevant text and promote the top-6 authoritative clauses to the LLM.